Efficient Data Management and Analysis in E-Commerce Using PostgreSQL

1st Sai Tanvith Gulla
Computer and Information Sciences
University at Buffalo, SUNY
saitanvi
50628597
saitanvi@buffalo.edu

2nd Akash Kumar Reddy Pallerla
Computer and Information Sciences
University at Buffalo, SUNY
apallerl
50628517
apallerl@buffalo.edu

3rd Harshith Reddy Peddireddy
Computer and Information Sciences
University at Buffalo, SUNY
hpreddie
50628547
hpreddie@buffalo.edu

***Abstract*—E-commerce applications generate huge volumes of transactional data that must be efficiently stored, retrieved, and analyzed for business optimization. The project targets the creation of a PostgreSQL-based online shopping database with assured scalability, data consistency, and query performance optimization.**

The system employs relational tables for critical business operations such as customer management, orders, payments, shipping, inventory tracking, and product reviews.The database offers easy structuring of the data, enabling efficient order processing, real-time inventory management, and high-security payment processing.

To provide best system performance, the project utilizes high-level SQL queries, index methods, and stored procedures. The products include a system relational schema, query optimization, sample data, and performance study report. A well-designed database is a good tool for e-commerce businesses, corporate efficiency, and data-based decision-making.

I. Problem Statement

The rapid growth in e-commerce has generated a flood of transactional data, which companies have to process and analyze efficiently. Legacy spreadsheet-based tools like Excel are not suited for handling scalability, complex querying, data integrity, and multiple concurrent access. The aim of this project is to develop a PostgreSQL-powered e-commerce database that can facilitate efficient order management, customer tracking, inventory control, and analytical processing.

With a relational database, querying will be faster, data consistency will be improved, and transaction processing will be performed, enabling smooth operations in an e-commerce environment. Such a database will enable major business operations such as order processing, product inventory tracking, and creating customer insights that are not feasible to accomplish within Excel due to its inability to handle large volumes of data and ensure data integrity.

A. Background and Significance

E-commerce sites generate hundreds of thousands of customer, order, payment, and inventory records. Excel is not appropriate to handle this volume of data due to:

- **Scalability Issues** – Excel has row limits and performance issues with large datasets.
- **Data Integrity Issues** – Spreadsheets can contain duplicate and inconsistent information that is not relationally structured.
- **Security and Concurrency** – Multiple users cannot update records simultaneously in Excel.
- **Query Performance** – Spreadsheets are inefficient for complex query aggregation, filtering, and joining operations.

Using a PostgreSQL-based relational database, we offer scalability, data consistency, and performance optimization, allowing businesses to make data-informed decisions from real-time insights.

B. Potential Contribution

The project contributes to the e-commerce sector by:

- **Improving Data Retrieval** – Optimized SQL queries improve operational effectiveness.
- **Maintaining Data Consistency** – Normalization (BCNF) minimizes redundancy and preserves integrity.
- **Enhancing Business Intelligence** – Structured data enables advanced analysis of customer patterns, sales trends, and stock control.
- **Easier Future Expansion** – The relational schema allows the database to scale up with increased transactions.

An efficient PostgreSQL e-commerce database establishes a strong foundation to handle sales, customer activity, and inventory while assisting firms in automating activities, enhancing customer service, and maximizing profitability.

II. Introduction

The rapid expansion of e-commerce has transformed the retail industry into a potential catalyst, enabling businesses to sell in international markets and offer seamless online shopping experiences. With the expansion of business, transactional volumes have also increased, and proper data handling is required. Spreadsheets, being traditional approaches, are never built to handle large amounts of data efficiently, which leads to inconsistency, redundancy, and delay in performance. An organized relational database is imperative to ensure data integrity, scalability, and real-time access to core business information.

The aim of this project is to implement and deploy an e-commerce database using PostgreSQL to support common business operations such as managing customers, managing orders, payment processing, tracking shipping, inventory management, and product feedback. The database is optimized for fast data retrieval, seamless data interaction among components, and maximum operational efficiency on e-commerce websites.

Through structuring a relational model, the system establishes a robust framework for storing, querying, and analyzing e-commerce information. The project uses advanced SQL queries, stored procedures, and indexing techniques to optimize query performance and effective handling of transactions. In addition, the system has real-time updating of inventory, tracking orders, and precise payment records.

The database system to be created shall offer a fast backend support for e-commerce websites with provision for auto activity, data consistency maintenance, and fast decision-making after end-to-end analysis. Installation of the system shall help improve organization within the data, boost query performance, and create an efficient, flexible solution which can easily accommodate ever-changing e-commerce website needs.

III. Target Users

A. Primary Users

1) Customers: Customers use the e-commerce website for product viewing, ordering, order tracking, and reviews. The database allows them to view products in stock, monitor purchase history, and be presented with special suggestions based on their history of purchases.

2) Business Owners and Managers: Business owners and managers use the database to analyze sales trends, view stock levels, and view revenue. They generate customer behavior reports and streamline business processes based on analysis.

3) Warehouse and Inventory Staff: Warehouse staff log into the system to track stock levels, restock products, and maintain stock records. Automatic alerts notify staff of low-level stock or demand changes.

4) Customer Support Representatives: Support representatives make use of the database in responding to customers' questions regarding orders, payments, and shipping. The system aids in identifying complaints, refunds, and purchase history to provide an excellent customer experience.

B. Database Administrators (DBAs) and IT Team

1) Who Takes Care of the Database?: The database administrator (DBA) ensures data security, query optimization, and backups. The IT team takes care of server uptime, prevents unauthorized access, and ensures smooth transactions.

C. Real-Life Scenario

Let's take "Walmart," an online consumer electronics shopping website. A user logs in, browses for a smartphone, and places an order. The database saves the transaction, updates inventory, and triggers a shipping request. The product is boxed by the warehouse staff, recorded as shipped by the system, and a tracking number is issued. Customer service uses the database to answer questions about order status. The business owner leverages analytics to track best-sellers and income patterns.

IV. List of Relations and Attributes

A. Customers Table

Purpose:

- Stores customer details like name, email, phone number, and address.
- Helps in customer authentication, order tracking, and account management.

Primary Key: CustomerID

- CustomerID (INT, PK, NOT NULL, AUTO_INCREMENT) – Unique customer identifier
- FirstName (VARCHAR(50), NOT NULL) – Customer's first name
- LastName (VARCHAR(50), NOT NULL) – Customer's last name
- Email (VARCHAR(100), UNIQUE, NOT NULL) – Customer's email for login
- PhoneNumber (VARCHAR(15), NULL) – Contact number
- Address (TEXT, NOT NULL) – Customer's address
- IsActive (BOOLEAN, DEFAULT TRUE) – Account status

B. Orders Table

Purpose:

- Records customer purchases, including the order status, total amount, and payment details.
- Links to Customers, Payments, and Shipping to manage the full transaction lifecycle.

Primary Key: OrderID
Foreign Key: CustomerID → Customers(CustomerID)

- OrderID (INT, PK, NOT NULL, AUTO_INCREMENT) – Unique order identifier
- CustomerID (INT, FK, NOT NULL) – Linked customer
- OrderDate (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP) – Date order was placed
- TotalAmount (DECIMAL(10,2), NOT NULL) – Total price of the order
- PaymentStatus (VARCHAR(20), DEFAULT 'Pending') – Payment status (Pending, Completed, Failed)
- ShippingStatus (VARCHAR(20), DEFAULT 'Processing') – Status (Processing, Shipped, Delivered)

Foreign Key Constraints: ON DELETE CASCADE (Deletes orders if a customer is removed)

C. OrderItems Table

Purpose:

- Stores each product within an order, including quantity and purchase price.
- Helps in inventory management by tracking sold items.

Primary Key: OrderItemID
Foreign Keys: OrderID → Orders(OrderID), ProductID → Products(ProductID)

- OrderItemID (INT, PK, AUTO_INCREMENT) – Unique item identifier
- OrderID (INT, FK, NOT NULL) – Order reference
- ProductID (INT, FK, NOT NULL) – Product reference
- Quantity (INT, NOT NULL) – Number of items purchased
- PriceAtPurchase (DECIMAL(10,2), NOT NULL) – Price per unit when purchased
- Subtotal (DECIMAL(10,2), GENERATED AS (Quantity * PriceAtPurchase)) – Calculated field

D. Products Table

Purpose:

- Maintains a catalog of all products, including descriptions, prices, and stock levels.
- Helps in displaying products to customers and managing stock availability.

Primary Key: ProductID

- ProductID (INT, PK, AUTO_INCREMENT) – Unique product identifier
- StockCode (VARCHAR(20), UNIQUE, NOT NULL) – Internal stock code
- Name (VARCHAR(100), NOT NULL) – Product name
- Category (VARCHAR(50), NOT NULL) – Product category
- Brand (VARCHAR(50), NULL) – Brand of the product
- Description (TEXT, NULL) – Product details
- UnitPrice (DECIMAL(10,2), NOT NULL) – Selling price
- StockQuantity (INT, NOT NULL) – Available inventory
- AddedDate (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP) – When added to the database
- IsAvailable (BOOLEAN, DEFAULT TRUE) – Availability status

E. Cart Table

Purpose:

- Stores items added to a customer's shopping cart before checkout.
- Helps in managing a temporary order before payment and purchase confirmation.

Primary Key: CartID
Foreign Keys: CustomerID → Customers(CustomerID), ProductID → Products(ProductID)

- CartID (INT, PK, AUTO_INCREMENT) – Unique cart identifier
- CustomerID (INT, FK, NOT NULL) – Customer who owns the cart
- ProductID (INT, FK, NOT NULL) – Product added to the cart
- Quantity (INT, NOT NULL) – Number of units added
- AddedDate (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP) – When item was added

F. Payments Table

Purpose:

- Records payment transactions linked to orders.
- Stores payment methods, transaction status, and amount paid.

Primary Key: PaymentID
Foreign Key: OrderID → Orders(OrderID), CustomerID → Customers(CustomerID)

- PaymentID (INT, PK, AUTO_INCREMENT) – Unique payment identifier
- OrderID (INT, FK, NOT NULL) – Related order
- CustomerID (INT, FK, NOT NULL) – Related customer
- PaymentMethod (VARCHAR(50), NOT NULL) – Payment type (Credit Card, PayPal, Bank Transfer)
- TransactionDate (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP) – Payment timestamp
- AmountPaid (DECIMAL(10,2), NOT NULL) – Amount paid
- TransactionStatus (VARCHAR(20), DEFAULT 'Pending') – (Success, Failed)

G. Shipping Table

Purpose:

- Manages shipping details for orders, including carrier, tracking number, and delivery dates.
- Helps in tracking delivery progress.

Primary Key: ShippingID
Foreign Key: OrderID → Orders(OrderID)

- ShippingID (INT, PK, AUTO_INCREMENT) – Unique shipment identifier
- OrderID (INT, FK, NOT NULL) – Associated order
- TrackingNumber (VARCHAR(50), UNIQUE, NULL) – Tracking ID for shipment
- Carrier (VARCHAR(50), NULL) – Shipping service provider
- ShippingDate (TIMESTAMP, NULL) – Date shipped
- DeliveryDate (TIMESTAMP, NULL) – Estimated delivery date
- ShippingCost (DECIMAL(10,2), NULL) – Cost of shipping

H. Reviews Table

Purpose:

- Allows customers to leave reviews and ratings for purchased products.
- Helps in customer feedback and improving product quality.

Primary Key: ReviewID
Foreign Keys: CustomerID → Customers(CustomerID), ProductID → Products(ProductID)

- ReviewID (INT, PK, AUTO_INCREMENT) – Unique review identifier
- CustomerID (INT, FK, NOT NULL) – Reviewer
- ProductID (INT, FK, NOT NULL) – Reviewed product

- Rating (INT, CHECK (Rating BETWEEN 1 AND 5)) – 1 to 5 stars
- ReviewText (TEXT, NULL) – Customer feedback
- ReviewDate (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP) – Date submitted

I. Discounts Table

Purpose:

- Stores discount codes and promotional offers applied during checkout.
- Helps in applying discounts and tracking promotions.

Primary Key: DiscountID

- DiscountID (INT, PK, AUTO_INCREMENT) – Unique discount identifier
- Code (VARCHAR(20), UNIQUE, NOT NULL) – Discount code
- DiscountPercentage (DECIMAL(5,2), NOT NULL) – Discount applied
- ValidFrom (DATE, NOT NULL) – Start date
- ValidUntil (DATE, NOT NULL) – Expiration date

J. Inventory Table

Purpose:

- Tracks stock levels for products and records restocking details.
- Helps in inventory management and avoiding stock shortages.

Primary Key: InventoryID
Foreign Key: ProductID → Products(ProductID)

- InventoryID (INT, PK, AUTO_INCREMENT) – Unique inventory entry
- ProductID (INT, FK, NOT NULL) – Product reference
- StockAdded (INT, NOT NULL) – Number of units restocked
- StockRemaining (INT, NOT NULL) – Current stock after update
- RestockDate (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP) – Date of stock update

K. Suppliers Table

Purpose:

- Stores supplier details, including contact information and company details.
- Helps in managing product sourcing and supply chain operations.

Primary Key: SupplierID

- SupplierID (INT, PK, AUTO_INCREMENT) – Unique supplier identifier
- CompanyName (VARCHAR(100), NOT NULL) – Supplier name
- ContactPerson (VARCHAR(50), NOT NULL) – Contact representative
- PhoneNumber (VARCHAR(15), NOT NULL) – Contact number
- Email (VARCHAR(100), NOT NULL) – Supplier email
- Address (TEXT, NOT NULL) – Supplier address

L. Customer Support Tickets Table (For Handling Customer Complaints & Queries)

Purpose:

- Stores customer complaints and queries for better issue resolution.
- Helps in customer support management by tracking complaints, responses, and resolutions.

Primary Key: TicketID
Foreign Keys: CustomerID → Customers(CustomerID)

- TicketID (INT, PK, AUTO_INCREMENT) – Unique ticket ID
- CustomerID (INT, FK, NOT NULL) – Customer raising the issue
- IssueCategory (VARCHAR(50), NOT NULL) – Type of issue (Order Issue, Payment Problem, Technical)
- Message (TEXT, NOT NULL) – Customer's query or complaint
- Status (VARCHAR(20), DEFAULT 'Open') – (Open, In Progress, Resolved)
- CreatedAt (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP) – Ticket creation date

V. Functional Dependencies and BCNF Decomposition

A. Step 1: Functional Dependencies for Each Relation

For each table, we define its functional dependencies (FDs):

B. Customers (BCNF)

FDs:

- CustomerID → FirstName, LastName, Email, PhoneNumber, Address, IsActive

BCNF Check: Already in BCNF (CustomerID uniquely determines all other attributes)

C. Orders (BCNF)

FDs:

- OrderID → CustomerID, OrderDate, TotalAmount, PaymentStatus, ShippingStatus
- CustomerID → Multiple Orders (1:M relationship)

BCNF Check: Already in BCNF (OrderID uniquely determines all attributes)

D. OrderItems (BCNF)

FDs:

- OrderItemID → OrderID, ProductID, Quantity, PriceAtPurchase, Subtotal
- (OrderID, ProductID) → Quantity, PriceAtPurchase, Subtotal (A composite key)

BCNF Check: Already in BCNF (OrderItemID is unique)

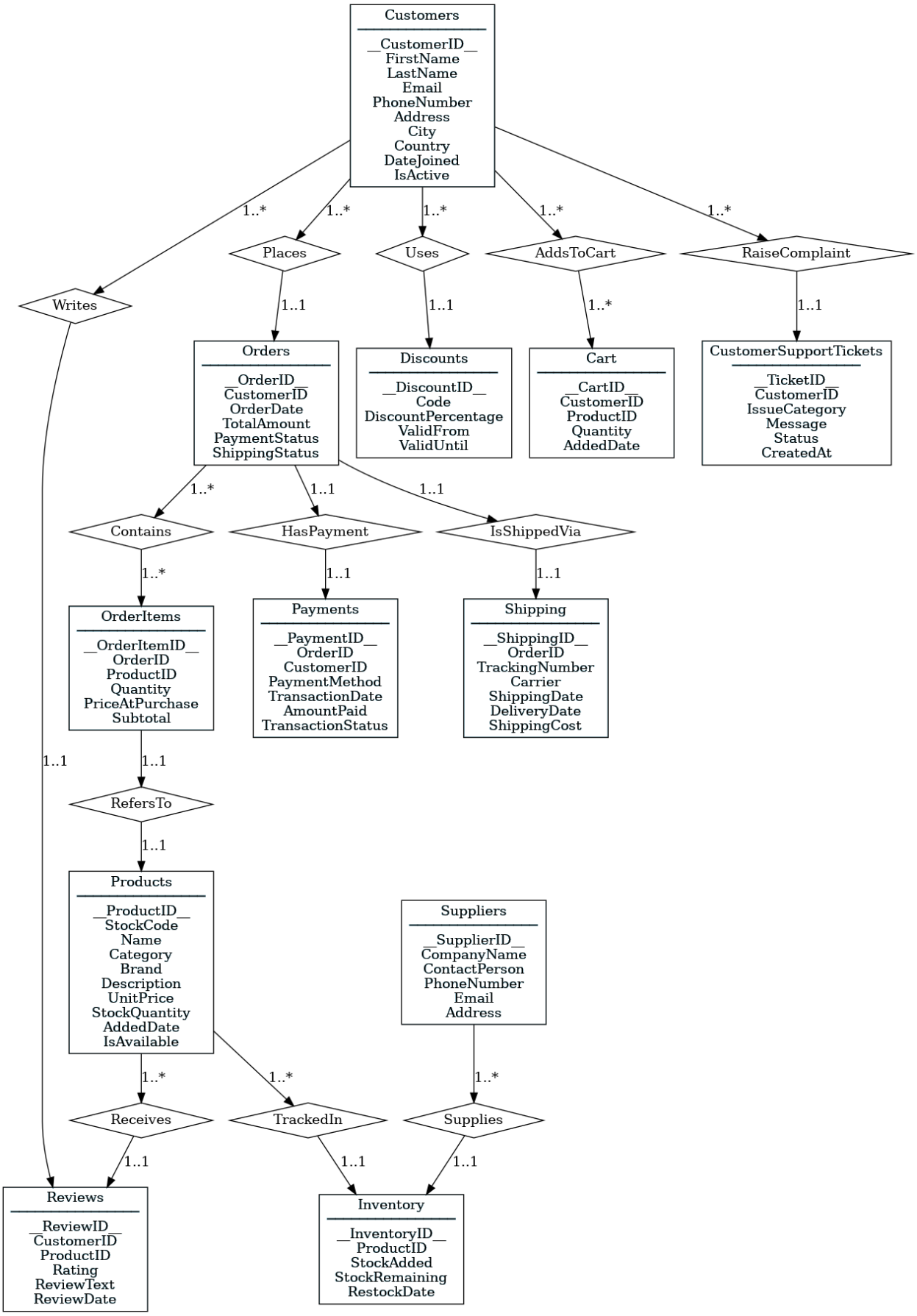

Fig. 1. Entity-Relationship Diagram of the PostgreSQL-Powered E-Commerce Database.

E. Products (BCNF)

FDs:

- ProductID → StockCode, Name, Category, Brand, Description, UnitPrice, StockQuantity, AddedDate, IsAvailable
- StockCode → ProductID (StockCode is unique to a product)

BCNF Check: Already in BCNF (ProductID is the sole determinant)

F. Payments (BCNF)

FDs:

- PaymentID → OrderID, CustomerID, PaymentMethod, TransactionDate, AmountPaid, TransactionStatus

BCNF Check: Already in BCNF (PaymentID uniquely determines all attributes)

G. Shipping (BCNF)

FDs:

- ShippingID → OrderID, TrackingNumber, Carrier, ShippingDate, DeliveryDate, ShippingCost

BCNF Check: Already in BCNF (ShippingID uniquely determines all attributes)

H. Reviews (Needs Decomposition)

FDs:

- ReviewID → CustomerID, ProductID, Rating, ReviewText, ReviewDate
- (CustomerID, ProductID) → Rating, ReviewText, ReviewDate (A customer can leave one review per product)

Violation: Not in BCNF

BCNF Decomposition:

- ReviewDetails (ReviewID, Rating, ReviewText, ReviewDate)
- ReviewMapping (CustomerID, ProductID, ReviewID)

I. Discounts (BCNF)

FDs:

- DiscountID → Code, DiscountPercentage, ValidFrom, ValidUntil

BCNF Check: Already in BCNF (DiscountID uniquely determines all attributes)

J. Inventory (BCNF)

FDs:

- InventoryID → ProductID, StockAdded, StockRemaining, RestockDate

BCNF Check: Already in BCNF (InventoryID uniquely determines all attributes)

K. Suppliers (BCNF)

FDs:

- SupplierID → CompanyName, ContactPerson, PhoneNumber, Email, Address

BCNF Check: Already in BCNF (SupplierID uniquely determines all attributes)

L. Cart (Needs Decomposition)

FDs:

- CartID → CustomerID, ProductID, Quantity, AddedDate
- (CustomerID, ProductID) → Quantity, AddedDate (A composite key)

Violation: Not in BCNF

BCNF Decomposition:

- CartDetails (CartID, Quantity, AddedDate)
- CartMapping (CustomerID, ProductID, CartID)

VI. Conclusion

We have created a robust e-commerce database to provide proper data management, integrity, and scalability in this project. This system makes it easy to manage customer transactions, orders, payments, shipping details, inventory control, product reviews, and customer support interactions efficiently.

Normalization of database schemas to Boyce-Codd Normal Form (BCNF) reduces redundancy. Cart and Reviews are two instances of non-BCNF table decomposition that traded information integrity for query efficiency.

Furthermore, through proper maintenance of real associations among clients, products, orders, and suppliers, the Entity-Relationship (ER) diagram offers a concise and organized approach to managing data.

Finally, this project demonstrates how a well-structured relational database becomes the hub of e-commerce activity with simplified transactions, secure data storage, and improved performance.

VII. Future Work

The e-commerce database was carefully designed with entity-relationship modeling to optimize efficiency, maintain data integrity, and establish both functional and foreign key dependencies. This structured development was fully normalized to Boyce-Codd Normal Form (BCNF) and implemented in Phase 1.

With the transition to Phase 2, our focus will shift to query efficiency, scalability, and transaction handling. The system's resilience will be tested through the execution of SQL queries with complex joins, subqueries, stored procedures, and triggers, ensuring its ability to recover from failures. Performance will be further optimized using indexing methods, and partitioning will support handling large datasets efficiently.

References

[1] [1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill, 2020, ch. 6, "Database Design Using the E-R Model," ch. 7, "Relational Database Design."